

```

import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /*
     * Complete the 'maxSubarray' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts INTEGER_ARRAY arr as parameter.
     */

    public static List<Integer> maxSubarray(List<Integer> arr) {
        int n = arr.size();

        // Kadane's algorithm for contiguous max subarray
        int maxEndingHere = arr.get(0);
        int maxSoFar = arr.get(0);

        for (int i = 1; i < n; i++) {
            maxEndingHere = Math.max(arr.get(i), maxEndingHere + arr.get(i));
            maxSoFar = Math.max(maxSoFar, maxEndingHere);
        }

        // Non-contiguous max subsequence
        int nonContigSum = 0;
        boolean hasPositive = false;
        int maxElement = arr.get(0);

        for (int x : arr) {
            if (x > 0) {
                nonContigSum += x;
                hasPositive = true;
            }
            maxElement = Math.max(maxElement, x);
        }
    }

```

```

    }

    if (!hasPositive) {
        nonContigSum = maxElement; // all numbers are negative
    }

    return Arrays.asList(maxSoFar, nonContigSum);
}
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        int t = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, t).forEach(tltr -> {
            try {
                int n = Integer.parseInt(bufferedReader.readLine().trim());

                List<Integer> arr = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split("
"))
                    .map(Integer::parseInt)
                    .collect(toList());

                List<Integer> result = Result.maxSubarray(arr);

                bufferedWriter.write(
                    result.stream()
                        .map(Object::toString)
                        .collect(joining(" "))
                        + "\n"
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
        bufferedReader.close();
        bufferedWriter.close();
    }
}

```



Problem Submissions Leaderboard Discussions Editorial Topics

You made this submission a few seconds ago.

Score: 50.00 Status: Accepted

People who solved The Maximum Subarray attempted this next:

Angry Children 2

Select K elements from N elements such that the sum absolute value of differences of adjacent elements is minimized

Solve Challenge

Submitted Code

Language: Java 8

Open in editor

```
1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.function.*;
8 import java.util.regex.*;
9 import java.util.stream.*;
10 import static java.util.stream.Collectors.joining;
11 import static java.util.stream.Collectors.toList;
12
13 class Result {
14
15     // Complete the function below
```

NEED HELP?

View discussions

View editorial

View top submissions